

Paractical:06

Aim: Write a simple CUDA program to print “HELLO WORLD”.

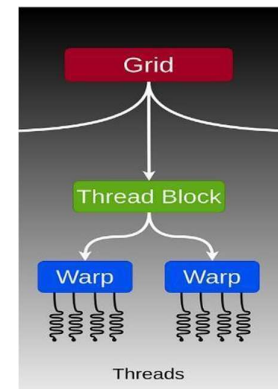
Theory:

Introduction To CUDA Programming

CUDA, an acronym for **Compute Unified Device Architecture**, is an advanced programming extension based on C/C++. Leveraging the capabilities of the Graphical Processing Unit (GPU), CUDA serves as a programming language that facilitates **parallel computing**. Developed by Nvidia, CUDA functions as both a parallel computing platform and an Application Programming Interface (API) model. By utilizing CUDA, individuals can effectively tap into the immense computational potential of Nvidia GPUs for various computing tasks, encompassing matrix processing and other linear algebra operations, beyond its conventional graphical calculations.

GPU logical layers

- **Grid:** A collection of blocks, representing the overall problem space to be solved.
- **Block:** A smaller group within a grid, containing multiple threads to handle subsets of data.
- **Warp:** A group of 32 threads within a block, executing instructions simultaneously.
- **Thread:** The smallest computation unit, performing individual tasks in parallel.



CUDA Program execution steps:

1. Load data into CPU memory
2. Copy data from CPU to GPU memory – e.g.,
`cudaMemcpy(..., cudaMemcpyHostToDevice)`
3. Call GPU kernel using device variable – e.g., `kernel<<<>>> (gpuVar)`
4. Copy results from GPU to CPU memory – e.g.,
`cudaMemcpy(..., cudaMemcpyDeviceToHost)`
5. Use results on CPU

Program 1**Code 1:**

```
%%writefile p1.cu

#include<stdio.h>
#include<cuda.h>
#include<cuda_runtime_api.h>

__global__ void AddIntsCUDA(int *a, int *b) //Kernel Definition
{
    printf("\n Hello World From GPU..!!");
    *a = *a + *b;
}

int main()
{
    int a=27,b=25;
    int *d_a, *d_b; //Device variable Declaration

    //Allocation of Device Variables
    cudaMalloc((void **)&d_a, sizeof(int));
    cudaMalloc((void **)&d_b, sizeof(int));

    //Copy Host Memory to Device Memory
    cudaMemcpy(d_a, &a, sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, &b, sizeof(int), cudaMemcpyHostToDevice);

    //Launch Kernel
    AddIntsCUDA<<<1,1>>>>(d_a,d_b);

    //Copy Device Memory to Host Memory
    cudaMemcpy(&a, d_a, sizeof(int), cudaMemcpyDeviceToHost);

    printf("\n The answer is : %d ",a);

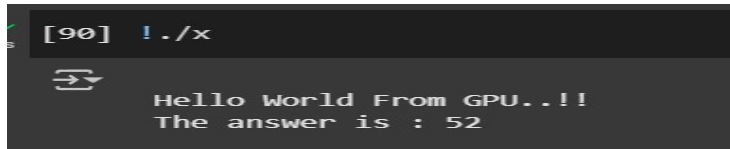
    //Free Device Memory
    cudaFree(d_a);
    cudaFree(d_b);

    return 0;
}
```

Step 1: change the runtime from cpu to gpu on runtime->runtime option

compile the program

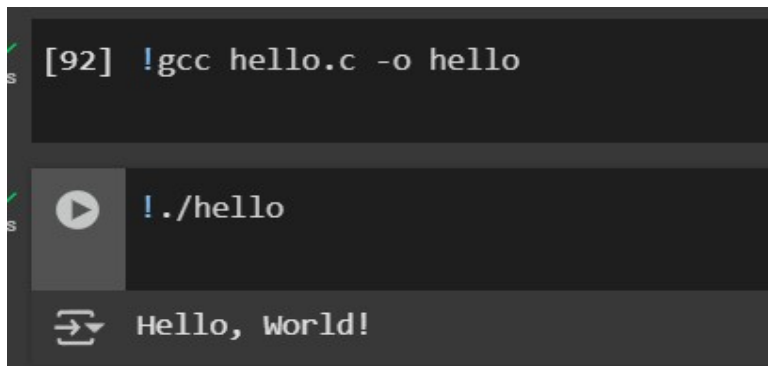
```
[89] !nvcc -arch=sm_75 p1.cu -o x
```

Step 2 : run the program

```
[90] !./x  
Hello World From GPU...!!  
The answer is : 52
```

Program 2**Code 2**

```
%%cuda  
#include<stdio>  
#include<iostream>  
_global_ void dkernel()  
{  
    printf("Hello world\n");  
}  
int main()  
{  
    dkernal<<<1,10>>>>();  
    cudaDeviceSynchronize();  
    return 0;  
}
```

Output:

```
[92] !gcc hello.c -o hello  
!  
!./hello  
Hello, World!
```

Conclusion:

In this practical, we successfully wrote and executed a simple CUDA program to print “HELLO WORLD”. The output demonstrated how each GPU thread runs the kernel in parallel, with multiple threads printing the message simultaneously. This experiment provided an introduction to CUDA kernel programming, familiarized us with the syntax of defining and launching kernels, and highlighted the difference between CPU serial execution and GPU parallel execution. It forms the basic foundation for learning more advanced CUDA applications such as parallel computation, vector addition, and matrix operations.

Paractical:07

AIM: write a simple CUDA program to add two numbers.

Theory:

CUDA (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA. It enables developers to utilize NVIDIA GPUs for general-purpose computing (GPGPU), accelerating computational tasks by leveraging thousands of parallel threads. CUDA provides APIs for writing programs in C, C++, and Python, allowing efficient execution of data-intensive operations.

Basic Cuda Functions

- **cudaMalloc()** – Allocates memory on the GPU device. `cudaMalloc((void**)&d_array, size);`
- **cudaMemcpy()** – Transfers data between CPU (host) and GPU (device). `cudaMemcpy(d_array, h_array, size, cudaMemcpyHostToDevice);`
- **cudaFree()** – Frees allocated GPU memory. `cudaFree(d_array);`
- **__global__** – Declares a kernel function that runs on the GPU.

```
__global__ void kernel_function() {  
    // GPU code  
}
```
- **cudaDeviceSynchronize()** – Ensures all GPU operations are completed before proceeding. `cudaDeviceSynchronize();`
- **cudaGetDeviceCount()** – Returns the number of available GPUs. `int count; cudaMemcpy(&count);`
- **cudaSetDevice()** – Selects a specific GPU for execution. `cudaSetDevice(0);`

Program:

```
%%writefile p1.cu

#include <stdio.h>
#include <cuda.h>
#include <cuda_runtime_api.h>

__global__ void AddIntsCUDA(int *a, int *b)
{
    *a = *a + *b;
}

int main()
{
    int a = 27, b = 25;
    int *d_a, *d_b;

    // Allocate memory on GPU
    cudaMalloc((void **)&d_a, sizeof(int));
    cudaMalloc((void **)&d_b, sizeof(int));

    // Copy data from CPU to GPU
    cudaMemcpy(d_a, &a, sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, &b, sizeof(int), cudaMemcpyHostToDevice);

    // Launch Kernel
    AddIntsCUDA<<<1,1>>>>(d_a, d_b);

    // Wait for GPU to finish
    cudaDeviceSynchronize();

    // Copy result back to CPU
    cudaMemcpy(&a, d_a, sizeof(int), cudaMemcpyDeviceToHost);

    // Output both numbers and result
    printf("First number : %d\n", 27);
    printf("Second number: %d\n", 25);
```

Output:

```
// Output both numbers and result
printf("First number : %d\n", 27);
printf("Second number: %d\n", 25);
printf("Sum          : %d\n", a);

// Free GPU memory
cudaFree(d_a);
cudaFree(d_b);

return 0;
}
```

⇒ Overwriting p1.cu

Execution & Output:

```
[5] !nvcc -arch=sm_75 p1.cu -o x
```

```
[6] !./x
```

```
⇒ First number : 27
   Second number: 25
   Sum          : 52
```

Conclusion:

The CUDA program successfully demonstrated parallel computing by adding two numbers using GPU acceleration. It utilized essential CUDA functions such as memory allocation, data transfer, kernel execution, and synchronization. This experiment highlighted the efficiency of GPU-based computation for basic arithmetic operations, laying the foundation for more complex parallel processing tasks.

Paractical:08

AIM: write a CUDA program to add two arrays.

Theory:

CUDA (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA. It enables developers to utilize NVIDIA GPUs for general-purpose computing (GPGPU), accelerating computational tasks by leveraging thousands of parallel threads. CUDA provides APIs for writing programs in C, C++, and Python, allowing efficient execution of data-intensive operations.

blockIdx.x, threadIdx.x, blockIdx.y, threadIdx.y values

- **threadIdx** → Identifies a thread within a block
- **blockIdx** → Identifies a block within the grid

For arradd<<<6,1>>> (6 blocks, 1 thread per block)

- **blockIdx.x: 0 to 5**
- **threadIdx.x: always 0**
- **blockIdx.y, threadIdx.y: always 0** (since 1D grid)

Program to Add Two Arrays:

```
%%writefile add_arrays.cu

#include <stdio.h>
#include <cuda.h>
#include <cuda_runtime_api.h>

__global__ void addArrays(int *a, int *b, int *c, int n)
{
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n) {
        c[idx] = a[idx] + b[idx];
    }
}

int main()
{
    int n = 5; // size of arrays
```

```
int h_a[] = {1, 2, 3, 4, 5};
int h_b[] = {10, 20, 30, 40, 50};
int h_c[5];

int *d_a, *d_b, *d_c;

// Allocate memory on GPU
cudaMalloc((void **)&d_a, n * sizeof(int));
cudaMalloc((void **)&d_b, n * sizeof(int));
cudaMalloc((void **)&d_c, n * sizeof(int));

// Copy arrays from host to device
cudaMemcpy(d_a, h_a, n * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_b, h_b, n * sizeof(int), cudaMemcpyHostToDevice);

// Launch kernel with enough threads
int blockSize = 256;
int numBlocks = (n + blockSize - 1) / blockSize;
addArrays<<<numBlocks, blockSize>>>(d_a, d_b, d_c, n);

// Wait for GPU to finish
cudaDeviceSynchronize();

// Copy result back to host
cudaMemcpy(h_c, d_c, n * sizeof(int), cudaMemcpyDeviceToHost);

// Print result
printf("Array A: ");
for (int i = 0; i < n; i++) printf("%d ", h_a[i]);
printf("\n");

printf("Array B: ");
for (int i = 0; i < n; i++) printf("%d ", h_b[i]);
printf("\n");

printf("Sum:   ");
for (int i = 0; i < n; i++) printf("%d ", h_c[i]);
printf("\n");

// Free device memory
cudaFree(d_a);
cudaFree(d_b);
cudaFree(d_c);

return 0;
}
```


Execution & Output:

```
!nvcc -arch=sm_75 add_arrays.cu -o x

[12] !./x

⇒ Array A: 1 2 3 4 5
   Array B: 10 20 30 40 50
   Sum:    11 22 33 44 55
```

Conclusion:

In this practical, we implemented parallel addition of two arrays using CUDA. The program utilizes **CUDA threads** to perform element-wise addition efficiently on the GPU. By leveraging parallel processing, we significantly improve execution speed compared to sequential CPU execution, making CUDA ideal for handling large-scale computations in scientific and engineering applications.

Paractical:09

AIM: Analyze the code using Nvidia-profilers.

Theory:

what is profiler

a **profiler** is a tool used to help you analyze the performance of your applications to improve poorly performing code. Profiling and diagnostics tools help you diagnose memory and CPU usage and other application-level issues. With these tools, you can collect performance data while you run your application.

Introduction to nvprof

The nvprof profiling tool enables you to collect and view profiling data from the command-line. nvprof enables the collection of a timeline of CUDA-related activities on both CPU and GPU, including kernel execution, memory transfers, memory set and CUDA API calls and events or metrics for CUDA kernels. Profiling options are provided to nvprof through command-line options.

Profiling results are displayed in the console after the profiling data is collected, and may also be saved for later viewing by either nvprof or the Visual Profiler.

Features of nvprof :

- **Kernel Timing:** Measure the precise execution time of GPU kernels to identify performance bottlenecks and optimize computational tasks effectively.
- **CUDA API Tracing:** Track and log CUDA API calls to gain detailed insights into the application's interaction with the GPU, helping to identify inefficiencies.
- **Timeline Analysis:** Visualize the sequence of kernel executions, memory transfers, and API calls on a timeline, enabling developers to better understand the application's behavior and resource usage.
- **Performance Metrics:** Analyze critical GPU events and metrics such as occupancy, memory throughput, and instruction throughput to fine-tune application performance.
- **Data Export:** Export profiling data into formats compatible with advanced analysis tools like NVIDIA Visual Profiler or Nsight Systems for further optimization and debugging

cuda program and nv prof steps on it :

```
%%writefile cuda.cu
#include <stdio.h>
#include <cuda.h>
#include <cuda_runtime_api.h>

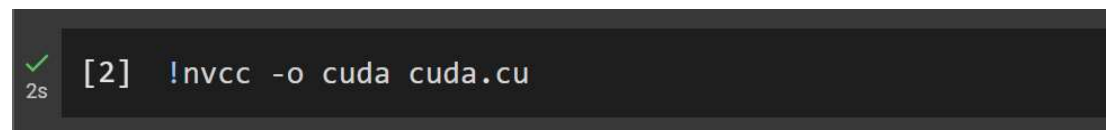
__global__ void kf()
{
    printf("Hello from GPU! Block %d, Thread %d\n", blockIdx.x, threadIdx.x);
}

int main()
{
    // Launch kernel with 2 blocks, 2 threads each
    kf<<<2,2>>>();

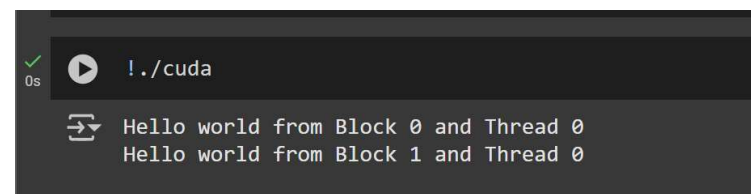
    // Wait for GPU to finish
    cudaDeviceSynchronize();

    // Force flush
    fflush(stdout);

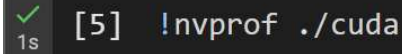
    return 0;
}
```

Step 1: compile the cuda program

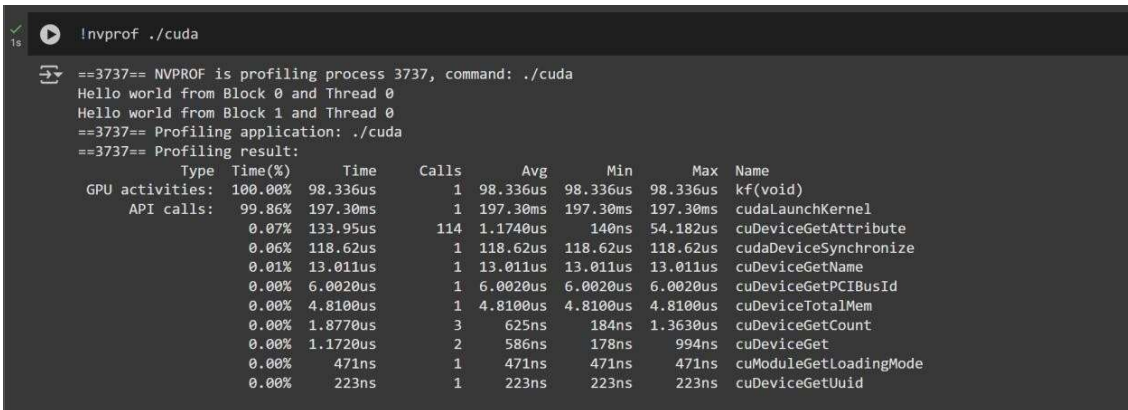
```
[2] !nvcc -o cuda cuda.cu
```

Step 2 : run the program

```
!./cuda
Hello world from Block 0 and Thread 0
Hello world from Block 1 and Thread 0
```

Step 3 : nvprof

```
[5] !nvprof ./cuda
```

Output:

```
==3737== NVPROF is profiling process 3737, command: ./cuda
Hello world from Block 0 and Thread 0
Hello world from Block 1 and Thread 0
==3737== Profiling application: ./cuda
==3737== Profiling result:
   Type      Time(%)      Time      Calls      Avg      Min      Max      Name
GPU activities: 100.00%  98.336us      1  98.336us  98.336us  98.336us  kf(void)
API calls:    99.86%  197.30ms      1  197.30ms  197.30ms  197.30ms  cudaLaunchKernel
              0.07%   133.95us     114  1.1740us   140ns   54.182us  cuDeviceGetAttribute
              0.06%   118.62us      1  118.62us  118.62us  118.62us  cudaDeviceSynchronize
              0.01%   13.011us      1  13.011us  13.011us  13.011us  cuDeviceGetName
              0.00%   6.0020us      1  6.0020us  6.0020us  6.0020us  cuDeviceGetPCIBusId
              0.00%   4.8100us      1  4.8100us  4.8100us  4.8100us  cuDeviceTotalMem
              0.00%   1.8770us      3    625ns   184ns   1.3630us  cuDeviceGetCount
              0.00%   1.1720us      2    586ns   178ns   994ns     cuDeviceGet
              0.00%    471ns      1    471ns   471ns   471ns     cuModuleGetLoadingMode
              0.00%    223ns      1    223ns   223ns   223ns     cuDeviceGetUuid
```

Conclusion:

Using nvprof, we analyzed the performance of the CUDA program. The profiler provided insights into kernel execution time, memory usage, and GPU utilization. It confirmed the successful launch of the kernel and highlighted the minimal overhead due to the simplicity of the program. This practical exercise demonstrated how nvprof can be used to debug and optimize CUDA applications by identifying potential bottlenecks and improving GPU performance.

Paractical:10

AIM: Demonstration of openMP and pthread functions.

Theory:

what is OpenMP

OpenMP (Open Multi-Processing) is a widely used standard Application Programming Interface (API) that allows programmers to write code that can run in parallel across multiple cores or processors on a shared memory system

Different functions of OpenMp

- **#pragma omp parallel:** Starts a parallel region where multiple threads execute concurrently. This is the foundation of parallel execution in OpenMP.
- **#pragma omp parallel for:** Parallelizes loops by dividing iterations among threads. It's one of the most commonly used directives to parallelize computationally intensive loops.
- **#pragma omp critical:** Protects a section of code from being accessed simultaneously by multiple threads to avoid race conditions.
- **#pragma omp barrier:** Synchronizes all threads, ensuring they reach a common point before continuing. This is important when threads need to work together or wait for each other.
- **#pragma omp reduction:** Performs thread-safe reductions (like sum, min, max) across threads, ensuring correct results when combining data.

Simple C program to demonstrate OpenMP

CODE:

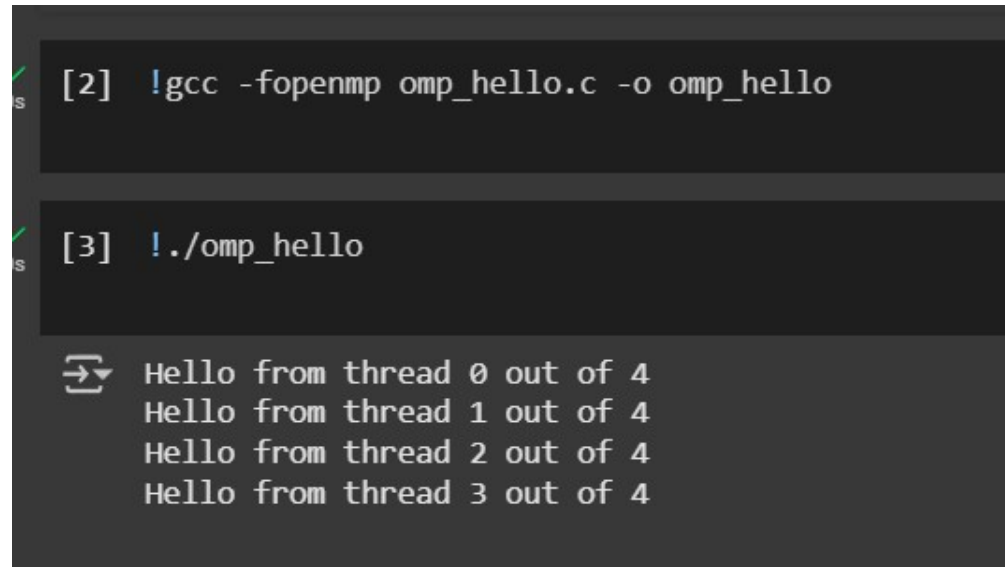
```
%%writefile omp_hello.c
#include <stdio.h>
#include <omp.h>

int main()
{
    // Set number of threads
    omp_set_num_threads(4);

    // Parallel region
    #pragma omp parallel
    {
        int id = omp_get_thread_num();    // thread ID
        int total = omp_get_num_threads(); // total threads
    }
}
```

```
    printf("Hello from thread %d out of %d\n", id, total);  
}  
  
return 0;  
}
```

Steps to Compilation and Execution output:



```
[2] !gcc -fopenmp omp_hello.c -o omp_hello  
  
[3] !./omp_hello  
  
Hello from thread 0 out of 4  
Hello from thread 1 out of 4  
Hello from thread 2 out of 4  
Hello from thread 3 out of 4
```

Conclusion:

This program demonstrates OpenMP's parallel execution by using the `#pragma omp parallel` directive to create parallel regions. The program prints "Hello, world!" from multiple threads in two parallel regions, with each thread displaying its unique thread number using `omp_get_thread_num()`. It showcases how OpenMP simplifies parallel programming by enabling concurrent execution of code, allowing multiple threads to perform tasks simultaneously, improving efficiency in multi-core systems.